

Contents

Control Your AI Mind	1
Foundational Lecture: From Chatbot To Research Agent OS	1
1. Opening Frame	1
2. Why AI Agents Became Visible Recently	2
3. Chatbot, Custom GPT/Gem, Cursor, And Agent: The Ladder	2
Level 1. Chatbot	2
Level 2. Custom GPT / Gem / Saved Instruction Bot	2
Level 3. IDE Assistant / Cursor-Like Workflow	2
Level 4. Terminal / Desktop / Workspace Agent	2
Level 5. Research Agent OS	3
4. Why Agents Matter For Economists And Researchers	3
5. The First Mindset: The Agent As A Companion	4
6. The Second Mindset: Ask The Agent	4
7. The Default Agent Is Blank	5
8. Memory: The Researcher's Library System	5
9. Rules: AGENTS.md, CLAUDE.md, GEMINI.md, And Soul Files	6
10. Skills: Repeated Workflow As Reusable Instruction	6
11. Tools, Plugins, MCPs, Hooks, Automations	7
12. Base Knowledge That Transfers Across Agents	8
13. Teaching Version: 20-Minute Opening Script	8
14. Applied Exercise	9

Control Your AI Mind

Foundational Lecture: From Chatbot To Research Agent OS

Corpus date: 2026-06-10

Input source: Huy's course-base monologue, synthesized into teaching-ready notes.

Use policy: This is Huy-owned teaching material. It should be treated as the base narrative for the next development phase.

1. Opening Frame

The first lesson should not begin with a tool. It should begin with the question learners actually have:

Why AI agents, and why now?

For many people, AI still means a chatbot: open ChatGPT, Claude, Gemini, or another chat interface; type a question; receive an answer; maybe upload a PDF or spreadsheet; maybe ask for a summary, draft, or code snippet. That workflow is useful, but it is still basically a conversation in a box. The AI responds to what the user places into the conversation.

An AI agent is different. The important shift is that the model is no longer only answering inside a chat window. It can work inside a computing environment: a folder, repository, terminal, browser, database, email/calendar workspace, or document system. It can read local project files, inspect data, write code, run scripts, edit documents, call tools, search sources, produce outputs, and preserve working state. In plain language:

A chatbot talks to you. An AI agent works with you inside your workspace.

This distinction is the foundation of the course.

2. Why AI Agents Became Visible Recently

AI agents did not appear from nowhere. Many pieces have existed for a while: GitHub Copilot, Cursor, Claude Code, Codex-like interfaces, Gemini/Antigravity-style IDEs, OpenCode, command-line agents, browser agents, and many experimental harnesses. Cursor had already shown that an AI assistant could understand and edit a codebase. Claude Code had already shown that a model could operate in the terminal. But for a long time, these tools were mostly visible to programmers and AI power users.

The recent shift is a convergence of three things.

First, the model capability became good enough. Frontier models became much stronger at long-context reasoning, code writing, tool use, and following multi-step instructions. A weak model with tools is dangerous and frustrating. A strong model with tools becomes a genuine worker.

Second, the interfaces became easier. Earlier agentic work often required terminal fluency or custom setups. Now users can access agents through desktop apps, IDEs, browser-integrated tools, Codex/Claude-like workspaces, and increasingly polished interfaces. A non-engineer can begin using agentic workflows without first becoming a systems engineer.

Third, the market narrative changed. When a tool or demo creates a public wave, people suddenly look for the broader category. In Vietnam, the recent excitement around agentic systems created a wave of curiosity; globally, OpenAI, Anthropic, Google, and open-source communities all accelerated agent releases around the same period. This created the feeling of a “storm” around AI agents.

The teaching point is: the idea is not brand new, but the adoption window is new. What used to be a power-user workflow is becoming a normal knowledge-work workflow.

3. Chatbot, Custom GPT/Gem, Cursor, And Agent: The Ladder

Learners need a ladder because otherwise every tool looks the same.

Level 1. Chatbot

A chatbot is a conversational interface. The user provides text, files, or screenshots; the model answers. It is strongest for explanation, summarization, brainstorming, drafting, and small code snippets. It is weak when the task requires sustained interaction with a project folder, repeated script execution, long audit trails, or multi-file changes.

Level 2. Custom GPT / Gem / Saved Instruction Bot

This is an upgraded chatbot with persistent instructions and sometimes uploaded knowledge. The user can define a role, tone, domain, and reusable prompt pattern. This is useful for repeatable conversational tasks: academic writing assistant, literature summarizer, grammar editor, or policy memo drafter. But it still usually depends on the user feeding the context into the chat.

Level 3. IDE Assistant / Cursor-Like Workflow

The AI enters the coding environment. It can read files, suggest edits, rewrite functions, and sometimes run commands. This is where users start feeling that the AI is no longer merely answering; it is participating in production.

Cursor is important historically because it made codebase-aware AI work feel natural. It sits close to modern agents because it understands project context and can edit multiple files.

Level 4. Terminal / Desktop / Workspace Agent

This is the current center of the course: Claude Code, Codex, Gemini/Antigravity-style agents, OpenCode, and similar systems. The agent works in a folder or workspace. It can:

- read project structure
- inspect documents
- write and edit code
- run scripts and tests

- produce PDF/HTML/CSV/figures
- call external tools
- use skills/plugins/MCP connectors
- maintain memory and project instructions
- execute multi-step plans

This is why agents matter for research. Empirical research is not a single answer. It is a workflow: source collection, cleaning, checking, plotting, writing, revising, packaging, and verifying.

Level 5. Research Agent OS

The final level is not one agent. It is an operating system around the agent:

- project instructions
- memory and wiki
- source atlas
- task contracts
- skills
- tool permissions
- verification loops
- logs and checkpoints
- templates and style guides
- public/private output policy

This is the destination of the course. Students are not only learning to use a tool. They are learning to build a system in which an agent can work safely and productively.

4. Why Agents Matter For Economists And Researchers

In Vietnam and in many research environments, AI adoption is still chatbot-heavy. People use ChatGPT, Claude, or Gemini to ask questions, summarize PDFs, translate paragraphs, draft emails, or write short code. That is useful, but it misses most of the leverage.

Research work is not just question-answering. It includes:

- collecting sources
- building a source map
- organizing papers and notes
- scraping web pages and APIs
- cleaning survey/admin data
- checking variable definitions
- generating descriptive tables
- running regressions
- debugging scripts
- writing memos
- producing slides
- reviewing drafts
- checking citations
- preparing replication packages

These are workflow tasks. Chatbots help with fragments. Agents can operate across the workflow.

For economists, this is especially important because much of the work is tedious but not intellectually trivial. A folder-to-figure workflow requires source judgment. A DiD regression requires estimand discipline. A literature review requires source boundaries. A policy memo requires claim calibration. The agent can accelerate production, but the researcher must keep control.

This is why the course is not “AI for coding”. It is “AI agents for research work”.

5. The First Mindset: The Agent As A Companion

The first mindset is that the AI agent should be treated as a long-term research companion, not a disposable answer machine. The word “assistant” is useful but incomplete. A good agent becomes closer to a working partner inside the research environment. It learns:

- who the user is
- what the user’s projects are
- what kind of writing the user prefers
- what research fields the user works in
- which tools the user uses
- how the user structures folders
- what standards the user expects
- what kinds of mistakes the user cares about
- what outputs the user considers acceptable

This does not mean the agent becomes human. It means the workflow becomes relational and cumulative. The more explicit the user is about preferences, standards, and project knowledge, the more useful the agent becomes.

The phrase for students:

You are not training the model globally. You are training the working relationship.

The agent improves because the environment improves: better instructions, better memory, better source maps, better skills, better examples, better feedback, better verification routines.

This creates a loop:

1. The user gives the agent better context.
2. The agent produces better outputs.
3. The user reviews and corrects the outputs.
4. The corrections become rules, memory, skills, or examples.
5. The next output improves.

The user develops too. The agent forces the user to articulate tacit knowledge. If a researcher wants the agent to review a paper well, the researcher must explain what a good review is. If a researcher wants the agent to clean data well, the researcher must explain units, keys, missingness, and sample rules. In this sense, the agent becomes a mirror for the researcher’s own workflow.

6. The Second Mindset: Ask The Agent

The second mindset is simple but powerful:

If you do not know how to do something, ask the agent how to do it.

This does not mean blindly obeying the answer. It means using the agent as the first interactive tutor for setup, syntax, workflow options, debugging, and conceptual explanation.

Examples:

- How do I set up this project folder?
- What should my AGENTS.md contain?
- What are the risks of giving this tool permission?
- How should I structure memory for this project?
- What should a DiD verification checklist include?
- How can I convert this recurring workflow into a skill?
- What files should I create before asking you to clean this dataset?
- What would make this output wrong?

This habit lowers the barrier to learning new agents. Once a student understands the base structure, they can move across Claude Code, Codex, Gemini, Cursor, OpenCode, or future agents by asking the agent itself to explain the local syntax and conventions.

The teaching warning is that “ask the agent” is not “trust the agent”. The learner should ask, compare, test, and verify. But asking quickly is how learners accelerate.

7. The Default Agent Is Blank

When a user opens a fresh agent in a blank folder, the agent may be powerful, but it is not yet personalized. It has the model’s general knowledge and tool access, but it does not know the user’s working style, project standards, data boundaries, preferred outputs, or long-term knowledge system.

The first serious step is personalization.

Personalization has three core layers:

1. **Memory.** What should the agent remember and retrieve?
2. **Rules / AGENTS.md / CLAUDE.md / GEMINI.md.** How should the agent behave in this project or with this user?
3. **Skills.** What recurring workflows should the agent execute in a standardized way?

Later layers include plugins, MCP connectors, hooks, automations, sub-agents, and external tools. But memory, rules, and skills are the foundation.

8. Memory: The Researcher’s Library System

Memory is not just “the chatbot remembers my name.” For research agents, memory is the way the agent stores and retrieves durable knowledge across sessions and projects.

There are many forms of memory:

- model-level memory
- project memory
- wiki/Obsidian memory
- source maps
- vector retrieval
- knowledge graphs
- session summaries
- progress files
- user profiles
- skill examples

The key teaching analogy is the library.

Imagine the user has one thousand books. If the agent needs an answer and must read all one thousand books every time, the workflow is slow, expensive, and unreliable. A real library does not work like that. It has catalogs, shelves, labels, call numbers, indexes, summaries, and librarians. When someone needs a book, the system points them to the right shelf and location.

That is how agent memory should work.

Raw files are the books. The wiki is the catalog and organized shelves. Source maps are the index. Retrieval systems are the librarian. The agent is the reader and worker. A good memory system tells the agent where to look, what is canonical, what is stale, what is private, and what can be used publicly.

The teaching rule:

Do not make the agent reread the entire library. Build a library system.

For Huy’s course, this naturally leads to a second-brain model:

- private corpus for full text, transcripts, screenshots, and raw notes
- wiki for compiled durable knowledge
- source map for citation and public/private status
- memory layer for fast retrieval
- project rules for how to use the above

This is the difference between “AI that remembers random facts” and “AI that works inside a structured research knowledge system.”

9. Rules: AGENTS.md, CLAUDE.md, GEMINI.md, And Soul Files

Rules files define how the agent should behave in a project or workspace. Different tools use different names: AGENTS.md, CLAUDE.md, GEMINI.md, project rules, workspace instructions, system prompts, or “soul” files. The names differ, but the purpose is similar.

The rule file answers two questions:

1. Who is the human and what does the human care about?
2. Who is the agent in this workspace and how should it work?

For a researcher, a strong rules file may include:

- user profile
- research domain
- preferred programming tools
- writing style
- evidence standards
- file structure conventions
- privacy boundaries
- tool order
- verification requirements
- output preferences
- what not to do

The point is not to make the agent “cute” or theatrical. The point is operational alignment. If the agent knows that the user is an applied economics researcher who cares about unit of observation, sample restrictions, fixed effects, clustering, weights, and claim discipline, the agent will work differently.

The team analogy is useful. Two people work better together when they understand each other’s roles, preferences, and standards. A rule file gives the agent that shared working context at the start of every session.

The warning: do not dump everything into rules. If a rule file becomes too long, stale, or vague, it can hurt more than help. Durable facts belong in a wiki or source map. Rules should contain operating principles, triggers, and stable preferences.

10. Skills: Repeated Workflow As Reusable Instruction

A skill is a reusable instruction bundle for a repeated workflow. It is not magic. It does not give the model a new brain. It tells the model how to handle a task consistently.

Simple example:

Instead of prompting every time:

“You are an academic writing expert. Use a formal but clear tone. Avoid hype. Preserve evidence discipline. Make claims precise. Do not overstate causal inference...”

The user creates an academic-writing skill. Then, when the task appears, the agent loads the instruction and follows the expected style.

For research, skills should be built around repeated workflows:

- academic writing
- paper summary
- literature map
- data audit
- Stata cleaning review
- regression table audit

- DiD verification
- source map creation
- referee report
- revision plan
- replication package check
- slide proofread
- policy memo review

A good skill has:

- trigger: when to use it
- inputs: what files or sources it expects
- process: the steps it should follow
- output format: what the result should look like
- checks: how it should verify itself
- ask-before-acting rules
- examples of good output

The teaching phrase:

A skill is how you turn tacit workflow knowledge into reusable agent behavior.

This is why skills are central to “Control Your AI Mind.” If the user repeats a workflow many times, they should not rely on memory or improvisation. They should encode the workflow.

11. Tools, Plugins, MCPs, Hooks, Automations

After memory, rules, and skills, learners can understand the broader ecosystem.

Agents become powerful because they can use tools. Tools can include:

- shell commands
- Python/R/Stata scripts
- file read/write
- browser automation
- search
- GitHub
- Google Drive
- Gmail/calendar
- databases
- APIs
- local apps
- PDF/Word/PowerPoint tools

Plugins and MCP-style connectors expand the agent’s reach. Hooks and automations let the system run checks, reminders, or routines at key moments. Sub-agents allow parallel work. Scheduled routines allow recurring workflows.

The teaching warning is that every tool increases both power and risk.

More tools mean:

- more capability
- more possible mistakes
- more permission questions
- more privacy concerns
- more need for logging and rollback

So the course should introduce tools only after students understand control. A student who cannot write a task contract should not start by connecting email, calendar, and databases.

12. Base Knowledge That Transfers Across Agents

The final point of the foundational lecture is that the exact tool matters less than the structure.

Claude Code, Codex, Gemini/Antigravity, Cursor, OpenCode, and future agents will differ in syntax, interface, pricing, and available plugins. But the base knowledge transfers:

- context window
- project folder
- permissions
- memory
- rules file
- skills
- tool calls
- source maps
- task contracts
- verification loops
- private/public boundaries
- logs and progress files

Once learners understand these, they can ask any new agent:

- What instruction file do you read?
- How do you handle memory?
- How do I define skills or reusable workflows?
- What tools can you use?
- What permissions do you have?
- How do I restrict you to this folder?
- How do I save progress?
- How do I verify your output?

This is the base launch of the course.

The user remains the same: their knowledge, standards, research taste, and judgment. The models and interfaces change. A good researcher learns how to carry their operating system across tools.

13. Teaching Version: 20-Minute Opening Script

Start:

“Most people know AI through chatbots. You ask, it answers. That is useful, but it is not the main shift. The main shift is that AI is moving from the chat window into your workspace. It can read your files, run your code, edit your drafts, inspect your folders, call tools, and produce artifacts. That is why we talk about AI agents.”

Then draw the ladder:

1. Chatbot
2. Custom GPT/Gem
3. IDE assistant
4. Workspace agent
5. Research Agent OS

Then give the research example:

“An economist does not only need an answer. They need a workflow: sources, data, code, figures, tables, writing, verification, and replication. A chatbot can help with pieces. An agent can work across the workflow. But the danger is that it can also produce plausible wrong outputs across the workflow. So the course is not about prompting. It is about control.”

Then introduce the two mindsets:

1. The agent is a long-term companion. You develop the working relationship through memory, rules, skills, feedback, and examples.

2. When you do not know how to do something, ask the agent. But do not trust blindly. Ask, test, verify, and encode what you learn.

Then introduce the three base layers:

1. Memory: the library system.
2. Rules: how the agent understands you and the project.
3. Skills: repeated workflows encoded as reusable instructions.

Close:

“If you understand these base layers, you can move across tools. Claude Code, Codex, Gemini, Cursor, OpenCode, or whatever comes next are different interfaces around the same basic problem: how to control an AI worker inside a knowledge workflow.”

14. Applied Exercise

Ask students to create a one-page “Agent Setup Blueprint” for themselves.

Required fields:

- Who am I as a researcher/worker?
- What tasks do I want an agent to help with?
- What must the agent never do without asking?
- What sources and folders can it read?
- What should it remember about me?
- What belongs in memory vs wiki vs raw source folder?
- What rules should be in AGENTS.md/CLAUDE.md?
- What three skills should I build first?
- What tools/plugins do I need later?
- What verification loop protects me from wrong output?

This exercise becomes the bridge from the opening lecture into the rest of the course.